OS - Operating Systems

User → OS Interface → System hardware
(Mac, iOS, Linux)

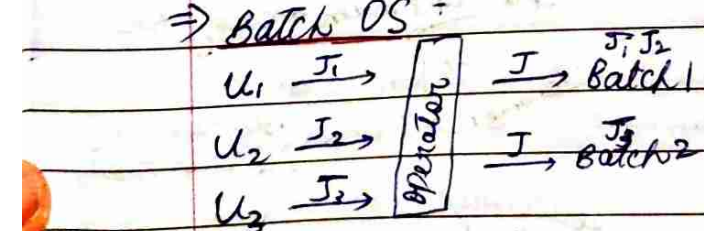
↳ OS Functions :

a) Process management
b) Memory management
c) File management
d) I/O Device management
e) Security

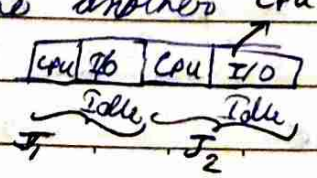
↳ Types of OS

a) Batch (Old technique)
b) Multiprogrammed
c) Multitasking
d) Clustered (1 aur 1 = 11) Joins comp. makes super comp
e) Distributed (Environment/schema not centered at point)
f) Embedded (Specific / narrow range functionality AC, Fan)

⇒ Batch OS :



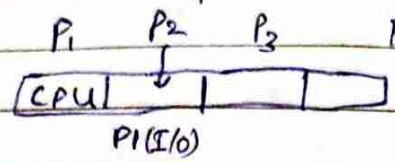
It combines similar jobs and assigns batches
To it, time is not defined. CPU utilizat^n is not optimal as tasks are performed one after the another CPU is idle



⇒ **Multiprogramming OS** -

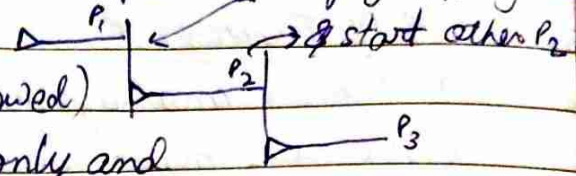
Focus on more than one process

No preemption nature [focus on more than 1 process]



Focus on more than one process

Preemption: forcefully stopping P1 & start other P2



⇒ Multitasking (Preemption allowed)

Here tasks are stopped only and ortaind resource & time is alloted to CPU and re-scheduled

⇒ **Multiprocessing OS** :

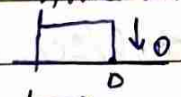
If we need many processors but no extra I/O & Memory

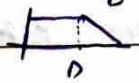
CPU 1 CPU2 Parallelism increased

P1 P2 Efficiency improved

Faster processing & Reliability

⇒ **Real time OS** : Time limit, no delay

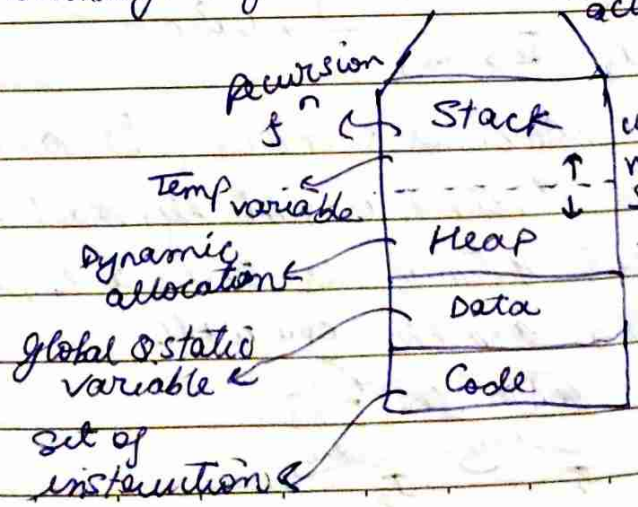
↳ Hard  strict

Soft  less strict, gradual zero

↳ **Process & Program**

CPU runs main memory files

Executable code → Program (Secondary memory) —Becomes→ Main memory → Process (dynamic obj) active entity



↳ Structure of OS : Components of OS are organised and interconnected. It is a simple structure

Application Program
↓
System program
↓
Device driver
↓
BIOS device driver

BIOS (Basic input output sys) Program directly interacting with the BIOS as there is no structure. Ex MSDOS

If user program fails the entire OS crashes

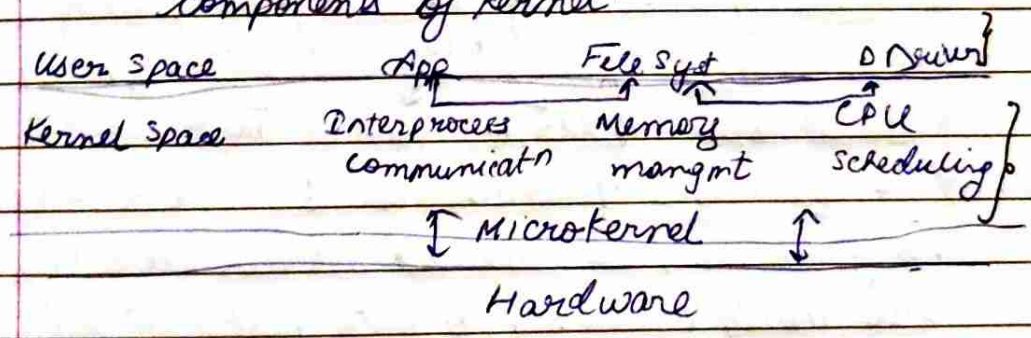
⇒ Layered :

Each layer has a specific task

Hierarchy: Layer can use the service provided by lower layers.

Independent Ex UNIX

↳ Micro kernel OS : Removing all non-essential components of kernel.



So if we want any user space application we dont have to disrupt the kernel space. Ex MacOS

It is complex as [Kernel + Interface = OS]

User → Interface → OS → Hardware

Interface → use of commands / use of GUI

↳ System calls are required by users to tell the OS to interact with the hardware and perform task. There are various calls for files, device, process mgmt.

↳ Mode → User mode (m = 1)
Applications operate here
Slave (unprivileged mode)
→ Kernel mode (m = 0) → OS operates
Core OS functionalities
Master / privelaged / System modes

⇒ User mode → Kernel mode: Execute system calls & return
Kernel mode → User mode: system calls to from kernel
context switching

↳ Fork System call: → 0 (Child process)
→ 1 (Parent process)

P() {
fork()
}
child parent

P() {
fork()
fork()
}
{child 1 Parent}
{child 2 child 1 Parent child 3}

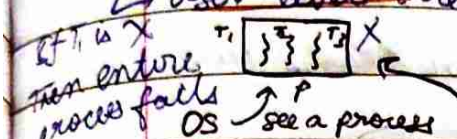
Increasing no. of workers to run

n → functions generate $2^n - 1$ Child & 1 parent
So y do we need child its because lets say there are multiple users, so with just a f^n call we can run the process. but maintaing child is expensive

↳ Threads: Lightweight process, we can add threads instead of creating child image of f^n. Here we are like adding hands from 2-6 for a worker efficiency increases and price is also same.
It is of 2 types single & Multithreading.
Context switching is faster when using threads.

↳ **Why do we need threads**: creating new thread in current process requires less time & effort than creating a new process. Threads can share common data, context switching is quick. Can be terminated in less time

↳ **User level thread** → Threads at user level, threads manged by user/applicat & faster



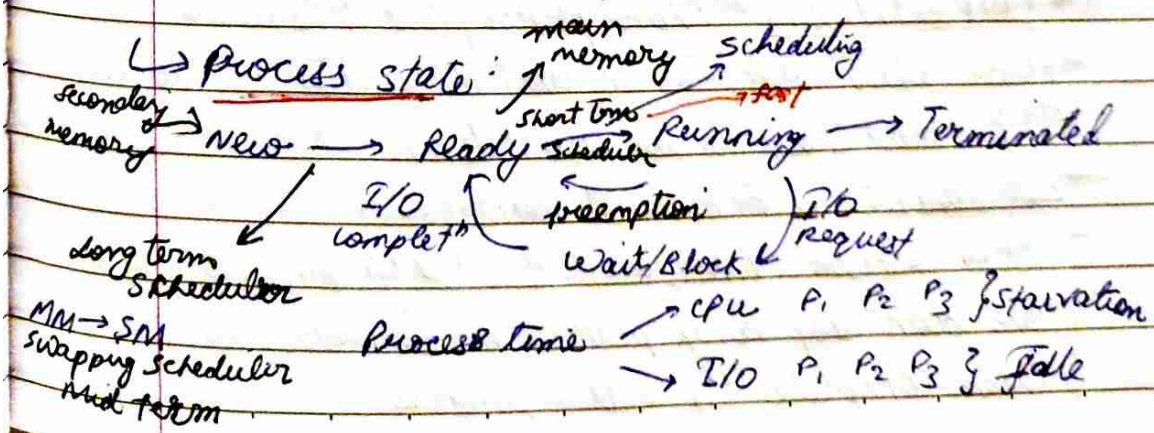
→ ULT gets blocked entirely (Process)

↳ **Kernel level thread** → Threads are managed by OS
→ No effect of one KLT to other
ULT < KLT << Process [Time]

↳ **Process control block**: Each process has a PCB
It contains all info regarding a process.

PID	
Process State	(new, ready, terminate states)
Program Counter	→ Points to next executable process
Priority	→ Each process has a priority
Register	→ Store the i/o values before context switching
File List	→ Interaction file list wrt P_1 or P_n
I/O info	→ Info of I/O devices is known
Protection	→ No 2 process interferes
.....	→ There can be addition field with diff OS

↳ **Process state**:



↳ Degree of multi programming :- Make continuous use of CPU we cant keep our CPU Idle, this is refered as degree of multiprogramming.
Long term scheduler (LTS) plays imp role in determing the degree of programming as it handles the SM ⇌ MM (secondary ⇌ main) memory processes

↳ CPU scheduling : Which (P) in ready queue is allotted to the CPU.

Preemptive	Non-preemptive
⇒ SRTF (Shortest remaing time first)	⇒ FCFS (First come first serve)
⇒ LRTF (Longest remaing " ")	⇒ SJF (Shortest job first)
⇒ Round Robin	⇒ LJF (Longest job first)
⇒ priority	⇒ HRRN (Highest response ratio next)

↳ Time quantum : ['z' units for each process (P)]
⇒ Arrival time : When P enters the ready state
Actual time to execute ⇒ Burst time : Time required by P to execute (Burst^n)
⇒ Waiting time : (TAT - BT) [Turn around time - Burst time]
Total time ⇒ Turn around time (TAT) : [CT - AT] Complet^n - Arrival time
⇒ Completion time : When P completes its execution
⇒ Response time : [P get CPU 1st time - AT]

↳ CPU scheduling :- Non-preemptive scheduling
→ FCFS : Starvation problem, last P executes at the end
→ SJF : Based on shortes burst time
→ LJF : Same as FCFS just reversed.
→ HRRN : Schedules based on (WT + BT) / BT from highest to ↓
We dont stop any process in between, we let them complete the entire process.

↳ CPU Scheduling: Preemptive scheduling
→ SRTF: Finds the shortest time at every unit & executes
→ LRTF: Just the opposite of SRTF works on ↑ Response time
→ Round robin: We use Queue and run a P for a order of min(Q, BT) and add P in Queue, say Q = 2.
→ Priority: We schedule tasks based on ↑ Priority

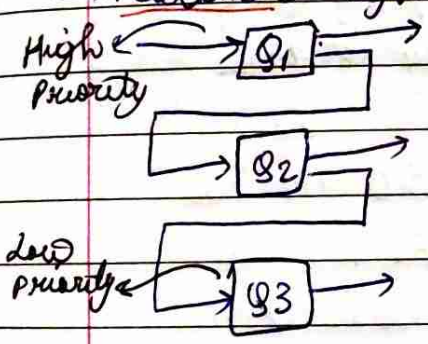
↳ Multilevel queue scheduling:

Priority ↓

System (P)	Priority	We make different
Interactive (P)	Round robin	queue for different
Batch (P)	SJF	types of Process (P)

And use diff scheduling algo.
We often see starvat^n as Batch (P) gets ↑ waiting time

↳ Feedback: Give priority to lower tasks as well



↓ demot^n

↑ Promotion

Exchange of multiple process is allowed in feedback. We can promote and demote process in diff queues.

↳ Process synchronization: Multiple processes can access shared resources without interfering with each other & maintain data consistency.
→ If result is inconsistent and if a data is shared amongst processes, then we have a race condition, when P_1 P_2 outputs wrong or data consistency is breached.
Entry section → Critical section → Exit Section

↳ Critical section problem:

→ Mutual Exclusion: At a time there can't be more that one (P) in critical sect^n

$P_1 \rightarrow \boxed{CS} \leftarrow P_2$

→ Progress: Interested process can go → CS (critical sect^n)
Not interested (P) should not stop interested (P)

→ Boundary waiting: Bound or limit should exist for a process to how many times a process can enter their CS (✓) 1 2 3 P_1
(X) 1 2 ... ∞ P_1

⇒ When P1 & P2 both of them cannot enter the CS (critical sect^n) we call it Deadlock as no process can be executed in the CS.

↳ Peterson's Solution: We can resolve all the mutual exclusion, progress & deadlock resolve
Use [Turn variable (T)] + [Flag variable] $\boxed{F|F}$ P_1 P_2
$T = 0, 1$

P_1	P_2
while (1){ $\boxed{F\|F}$ P_1 P_2	while (1){
F[0] = True	F[1] = True
Turn = 1	Turn = 0
while (Turn == 1 && F[1] == True);	while (Turn == 0 && F[0] == True);
CS	CS
F[0] = False	F[1] = False
NCS (Non critical section code)	NCS
}	}

↳ Semaphore: Binary & Counting semaphores
Managen concurrent process by use of a integer value (variable)

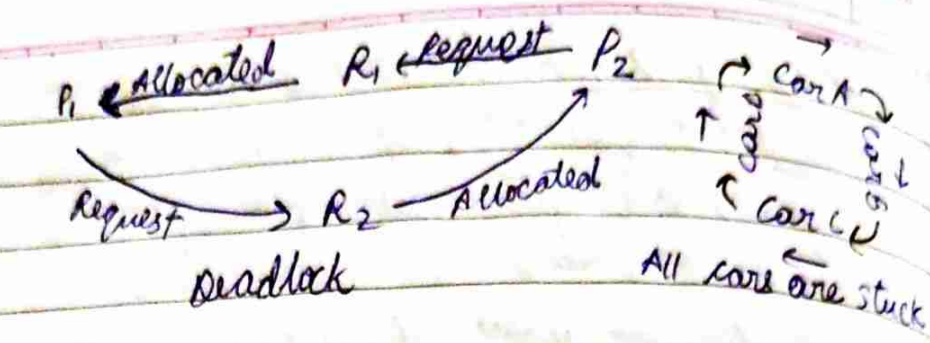
↳ **Binary semaphore**: (0 or 1) works on 2 functions
variable P(S) {wait, sleep or down} Same nature like
V(S) {signal, wake-up or up} entry exit code F,T

↳ **Counting semaphore**: ($-\infty$ to ∞) int variable value
S=5 (max value) P(S) [CS] V(S) S no. of
processes can exist at CS at a time.

↳ **Producer consumer problem**: Basically this is
a problem where producer adds an item
and consumer takes that item and this process
happen simultaneously. P [I I I] C
Semaphore → Full: 0 filled positions spaces
→ Empty: N empty spaces
→ Mutex: For mutual exclusion
Initially the buffer is empty, we start by calling Producer

```
Producer () {                          Consumer() {
  while (True) {                         while (True) {
    "produce an Item"                      wait (full);
P ← wait (Empty);  N→N-1                   wait (mutex);
mutual exclusn ← wait ( Mutex);            consume();
    Insert();                              Signal (mutex);
V ← signal (mutex);  N→N+1                 signal (empty);  N→N+1
    signal (full);                       }
  }                                    }
} ↑&↓ the space in item list
```

↳ **Deadlock**: All processes are stuck and cant be send
to the CS for execution, so we tweak some changes
by changing the order of any one process. Ex fork problem



↳ Conditions of deadlock:

- Mutual Exclusion: Only 1 P can use resource at a time
- Hold and wait: P_i holds min 1 resource & wait for P_j to aquire
- No preemption: P can release resource voluntarily after comp
- Circular wait: $P_1 \leftarrow P_2$ Dependent on each other

↳ Deadlock handling methods:

↳ Prevention: Its always better than cure. Deadlock should not happen "very strict".

⇒ Mutual exclusion: Not all resources can be made shareable

⇒ Hold & wait: Can violate it

Process should have all required resource before running, otherwise not allowed to run	To request for additional resources first release the resource that is being held	Time out on waiting
$P_1 \{ R_1, R_2, R_3 \}$ All must be present	$P_1 \rightarrow R_1$, R_2	$P_1 \rightarrow R_1$ (5 minutes)

⇒ No preemption: Preemption can be done based on priority, violate deadlock condition

Not allowed cos deadlock

$P_2 \leftarrow P_1 \rightarrow (R_1, R_2)$

$P_1 \leftarrow R_1$

$P_2 \leftarrow R_2$

⇒ Circular wait: Process can request resources in higher increasing order. P_2 can't req R_1

↳ Avoidance: If you cant prevent it, try avoid it. A bit less strict, we try to provide all info to System

⇒ **Banker's Algorithm**: Is a deadlock avoidance algorithm used in OS to safely allocate resources to multiple processes while ensuring that the system doesn't enter a deadlock state. This algorithm has basic maths where allocated ≥ need/requested, difference gives new availability and we repeat same equation.

→ Lets say if we get a deadlock we can terminate all P

→ Terminate each P and check for deadlock using prompt^n.

↳ **Ignorance [Ostrich algorithm]**: Rare situation

→ No concept of deadlock exists.

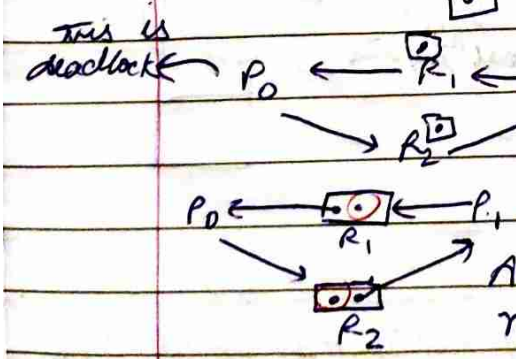
→ Windows, mac follow this approach to handle deadlock by ignoring it, does not count for deadlocks.

→ Its a rare event & handling cost of deadlock is high.

↳ **Resource allocation graph**:

node / vertex — This is how the resource is allocated.

- Process: P_i
- Resource: Single instance, Multi instance
- Edge: Assign ($R \rightarrow P$), Request ($P \rightarrow R$)



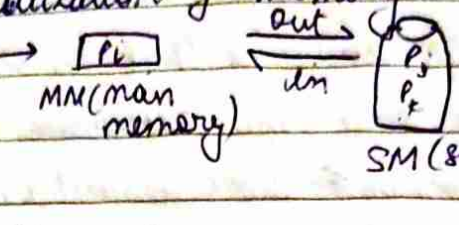
→ If cycle exists deadlock may/~~may not be~~ present

→ If cycle doesn't exist then deadlock doesn't exist.

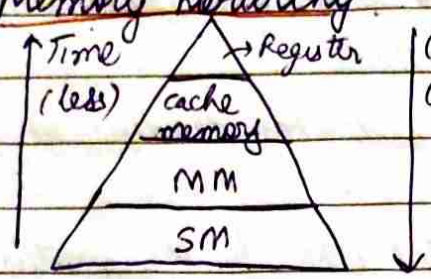
Available (1, 1) [P_0, P_1] No deadlock

multi instance

↳ **Memory Management**: Proper/efficient/optimal utilization of memory. Multiprogramming



↳ **Memory hierarchy**:



Degree of multiprogramming ∝ CPU utilization

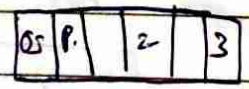
Process → CPU, → I/O

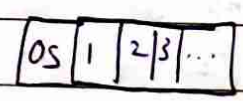
There are 'n' processes in MM, and CPU operat^n is 'm' times

∴ CPU utilization = $(1 - m^n)$

↳ **Memory management techniques**:

- Non-contiguous
  - Paging
  - Segmentat^n
  - Segmented paging
- Contiguous
  - Fixed Portion
  - Variable partition

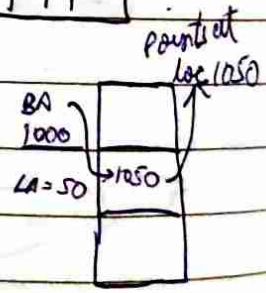




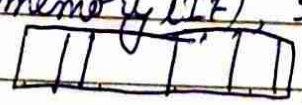
↳ **Contiguous [Address translation]**

PA = LA + BA → relocat^n register

Physical add. (O/P) — Logical add (Given by CPU) — Base address



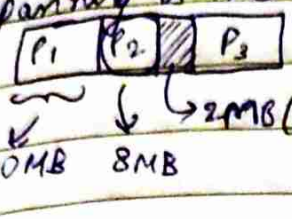
↳ **Fixed partitioning**: Fixed no. of partitions, partit^n size may be different, spanning not allowed, wastage of memory (IF), size limitat^n (not above max PA)



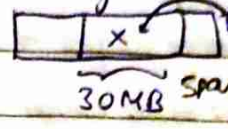
Unequal Partitions

Equal partit^n

→ Spanning is the expansion of memory when it is not allocated



Internal fragmentation

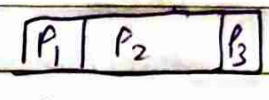


P_4 = 50MB cannot increase spanning the memory

→ Degree of multiprogramming is affected here

→ External fragmentation exists (have space but can't allocate to P) due to contiguous condition

↳ Variable partitioning:



→ Variable size partitions (dynamic nature)

→ No internal fragmentation but ✓ External fragmentation

→ No affect on DOM (degree of multiprogramming)

→ No restrict^n on process size.

→ To remove EF we need to do process shifting but expensive

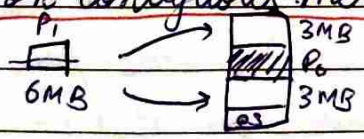
⇒ First fit: finds space from start and allocates

⇒ Next fit: finds from previous allocated memory

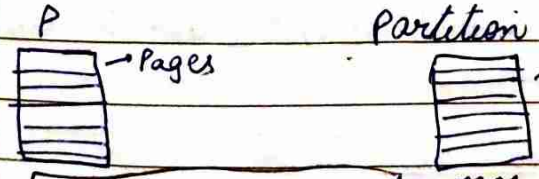
⇒ Best fit: less wastage of space (less IF)

⇒ Worst fit: Most wastage of space (more IF)

↳ Non-contiguous memory allocation: Fixes EF



Here we always need to analyse the no. of holes & their size to partition the process accordingly



One page fits in one frame

Page size = frame size

→ Logical address → Memory unit does this allocat^n → n^{th} bit of a page = frame

Page no (PNO)	offset	Translation →	Frame no (FNO)	Offset
2bit	2bit		3bit	2bit
P[0] = 0 0	00 → 1st Bit		F[0] 0 0 0	
P[1] = 0 1	01 → 2nd		F[1] 0 0 1	
P[2] = 1 0	10 → 3rd			
P[3] = 1 1	11 → 4th		F[7] 1 1 1	

↳ **Page table**: It is a data structure & every process has a separate PT stored in MM.

No. of pages = No of entries in PT

Page table entry size = No of frames

But we need to access 2 times page table & actual byte table

Storage overhead (To store PT), translat^n is complex task.

↳ **Page table entry**:

cache is closer to CPU

Frame number	Present Absent	Protect^n (Read/write)	Reference 1st time or not	Caching E/D	Dirty update

Mandatory part — Optional part

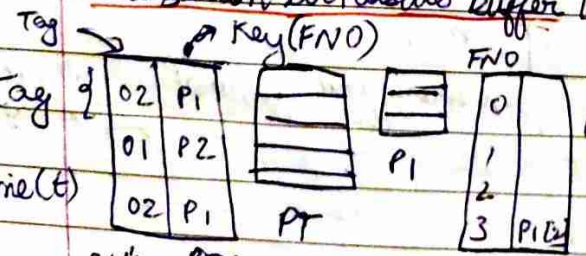
→ There is also multilevel paging where we breakdown the page table PT → $PT_{(1)}$ and store the components of it in various fram size → $PT_{(0)}[0]$, $PT_{(0)}[1]$ ...

↳ **Inverted paging**: Making global Page table (PT)

FNO	PNO	PID → Process ID
0	0	P_1
1	1	P_2
2	0	P_3
⋮	⋮	⋮
n	⋮	P_n

With this we just require a single global PT which holds info for individual frames - page no wrt its process ID. But we have to search the entire table for this.

↳ **Translation lookaside buffer [TLB]**:



We create a hashmap of key {PNO, PID} value {FNO}

→ Found [TLB hit]

→ Not found [TLB miss]

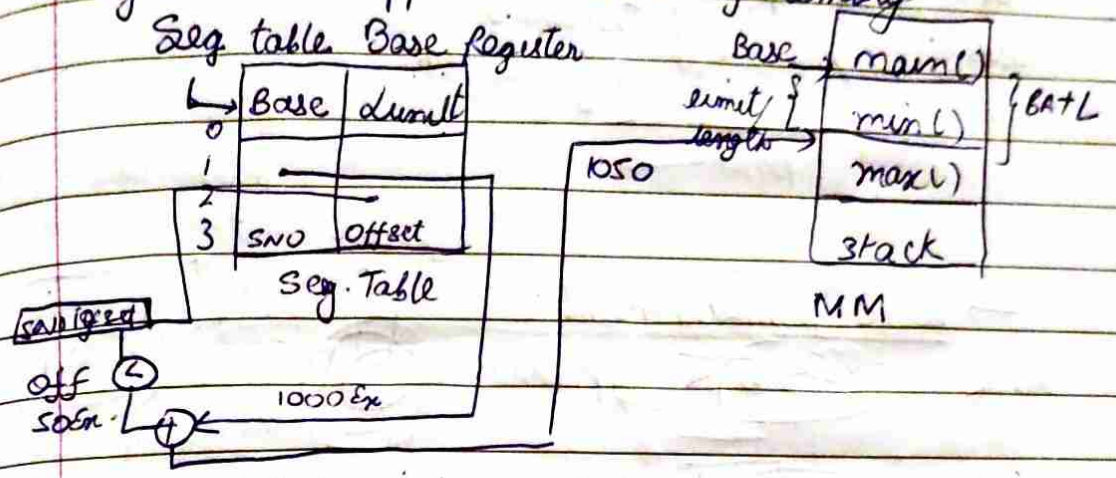
$T = \beta(t+m) + (1-\beta)(t+2m)$

TLB hit — TLB miss

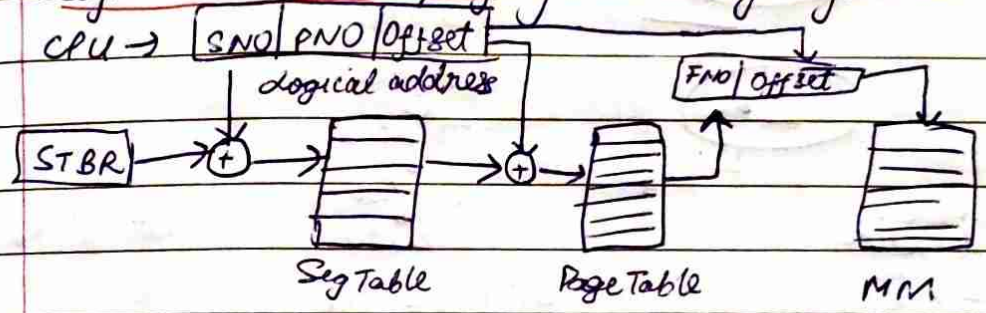
↳ **Segmentation**: It is similar to paging but the page will be of variable size, different sizes.

→ Paging was near to OS view but away from the User view.

→ Segmentation support User view of memory.



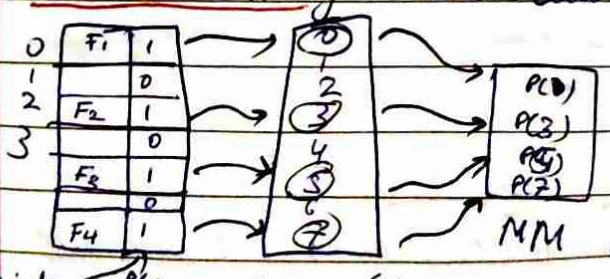
↳ **Segmentation with paging**: Dividing segments into pages.



↳ **Demand paging & page fault**: Don't load untill requested

CPU	OS takes	Again	If page not found
Request	required	send	in MM "page fault"
Page fault as sn page in SM	page in MM SM→MM	request as page in MM	
User CS	OS	CS User	

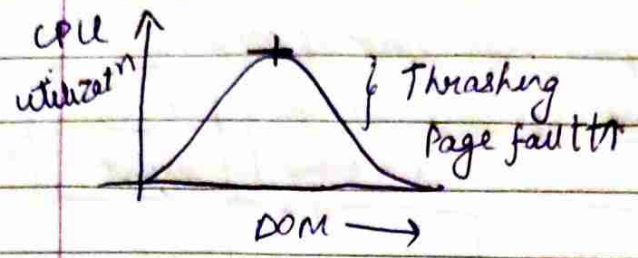
↳ **Virtual memory**: You can load part of process in MM



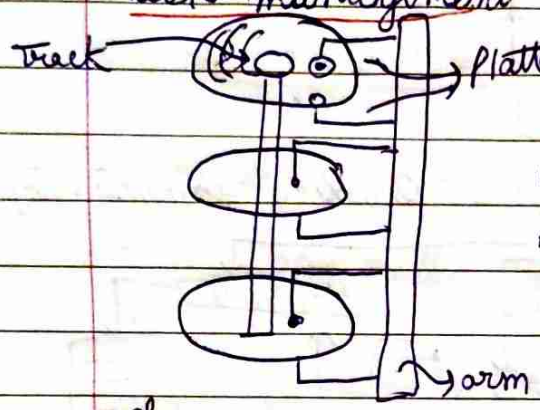
It can be assigned by
→ From allocation
→ Page replacement

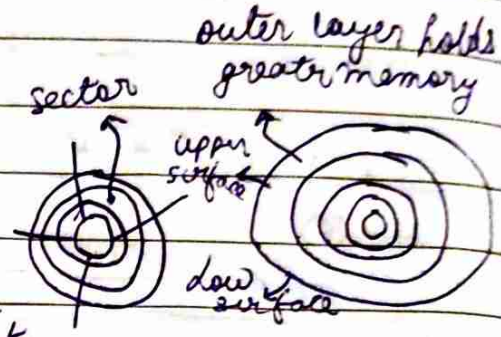
When page fault happens the process is shifted from SM→MM where space is free. If not free we check dirty bit (Not modified) 0/1

→ **Thrashing**: If we increase DOM, CPU utilizat^n ↑
But on higher DOM condit^n Page fault also ↑
which in turn reduces and overloads CPU utilizat^n



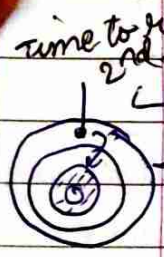
↳ **Disk management**:





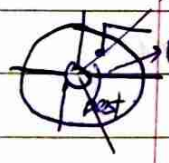
Disk
↓
Platter
↓
surface
↓
Track
↓
Sector
↓
Data

↳ **Disk acess time**: ST + RL + TT



→ **Seek time**: Time required for read/write head to move on target track $ST = n \times S$ (n: no. of track head move; S: ST/track)

→ **Rotational latency**: Time taken by r/w head to move to target sector $RL = \frac{A}{F}$ (A: angle b/w curr & target sector; F: Rotational frequency)



$ARL = \frac{1}{2}$ full rotat^n

→ **Transfer time**: The time taken to R/W the data
$TT = \frac{B}{R}$ (B: Size of data in bytes; R: Data transfer rate)
[No. of heads × Capacity of track × No. of rotat^n in 1 sec]

↳ **Disk scheduling algorithm**:

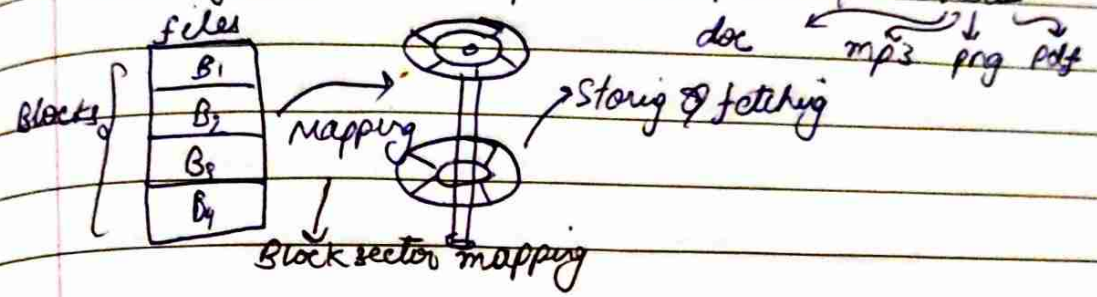
→ FCFS: Iterates the track in order of track list first come

→ SSTF: Shortest time sorts the track list & reaches closest

→ SCAN: Move to smaller or larger values, go till extreme end.

→ LOOK: Same as SCAN but dont go to extreme end.
→ C-Scan: Start to end → 0 → find tracks. Covers entire track.
→ C-LOOK: Move till extreme end and return lowest track no.

↳ File system: Users operate/work/interact the Files

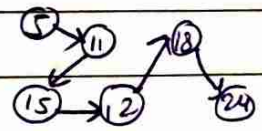


→ File operations: Create, read, write, truncate, delete
→ "Get info": Name, size, type, create date, last modify, permissions

↳ Allocation methods: → Contiguous
→ Non contiguous → Linked list
→ Indexed

→ Disk utilization ↑
→ Accessibility ↑
→ Avoid external fragmentat^n
→ Linked list

File	Start	end
X	5	24

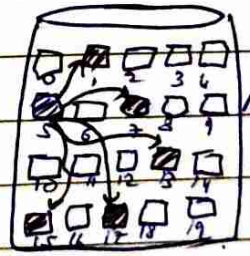


No Ext. fragmentat^n
↑ Seek time

→ Indexed file:

Folder / Directory

File	Index Block
X	5



No Ext. fragmentat^n
Random access easy

Stores all the pointer from 5 in index block